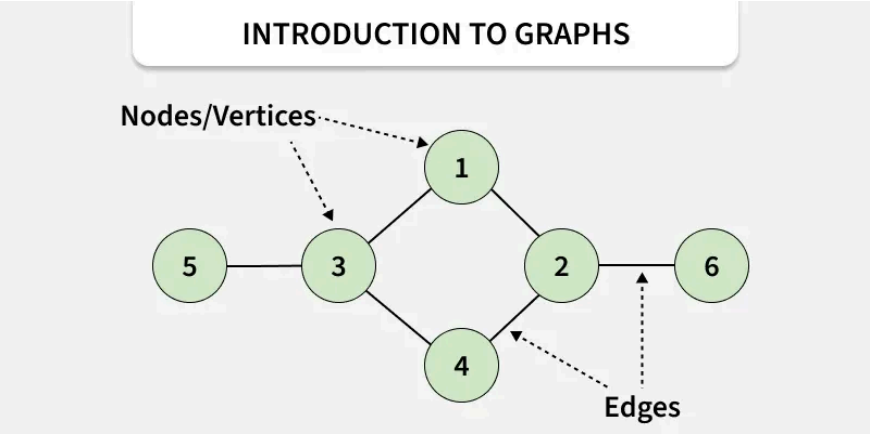
Graph

A **graph** is a **non-linear data structure** made up of **vertices** (nodes) and **edges** (connections) that represent relationships between objects. Unlike arrays or linked lists, graphs do not follow a sequential order.

Example: On a map, each city is a vertex, and each road connecting two cities is an edge. This way, a graph represents how cities are linked.



Components of Graph Data Structure

- **Vertices:** Vertices are the fundamental units of the graph. Sometimes, vertices are also known as vertex or nodes. Every node/vertex can be labeled or unlabelled.
- **Edges:** Edges are drawn or used to connect two nodes of the graph. It can be ordered pair of nodes in a directed graph. Edges can connect any two nodes in any possible way. There are no rules. Sometimes, edges are also known as arcs. Every edge can be labelled/unlabelled.

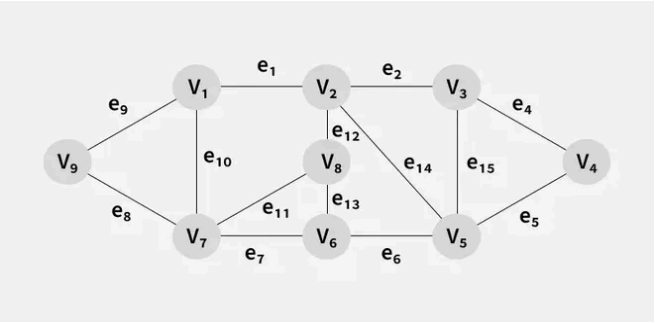
Types Of Graphs in Data Structure and Algorithms

A graph can be divided into multiple categories based on different properties such as edges, direction, connectivity and many others.

Based on weight :

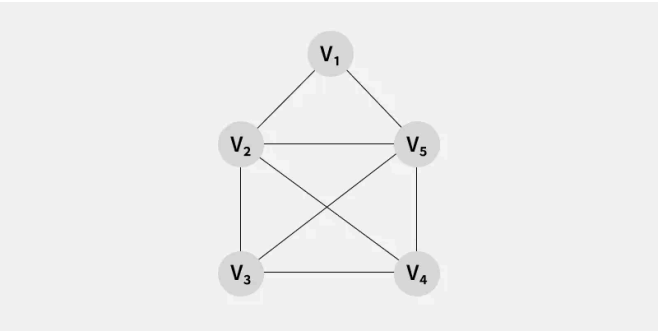
1. Weighted Graphs

A weighted graph is a graph where each edge has a number (weight) that represents distance, cost, or time.



2. Unweighted Graphs

An unweighted graph is a graph where all edges are treated equally, with no extra values like distance or cost.



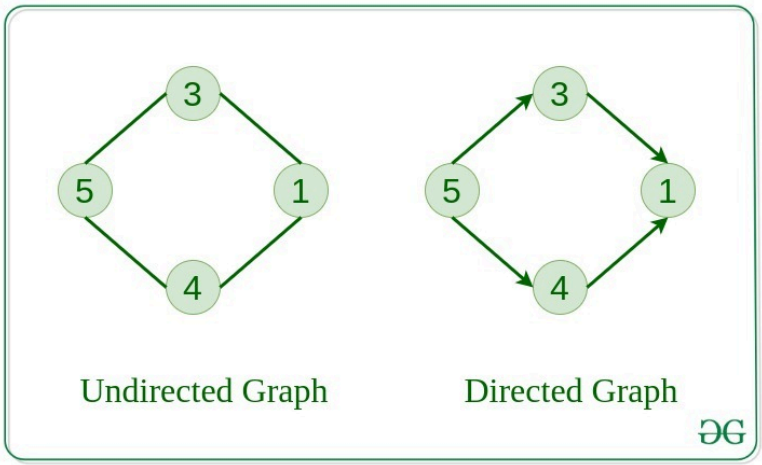
Based on Edge Direction:

3. Undirected Graph

A graph in which edges do not have any direction. That is the nodes are unordered pairs in the definition of every edge.

4. Directed Graph

A graph in which edge has direction. That is the nodes are ordered pairs in the definition of every edge.



Representation of Graph Data Structure:

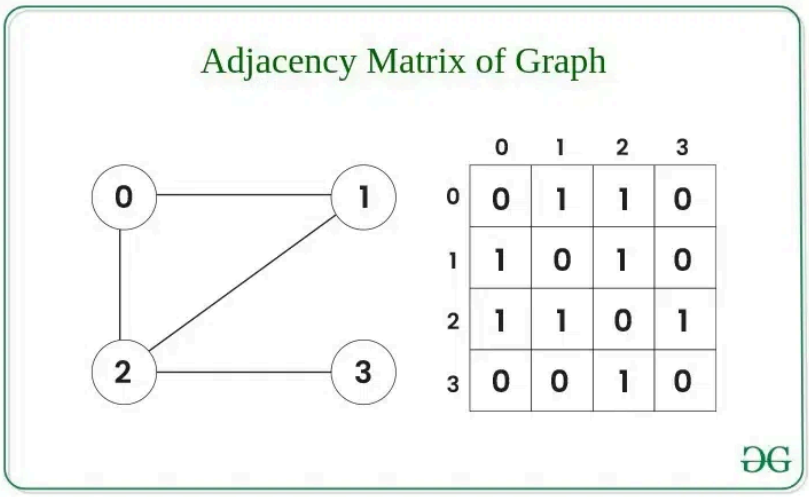
There are multiple ways to store a graph: The following are the most common representations.

- **Adjacency Matrix**
- **Adjacency List**

Adjacency Matrix Representation of Graph Data Structure:

In this method, the graph is stored in the form of the **2D matrix** where **rows and columns denote vertices**. Each entry in the matrix represents the **weight of the edge between those vertices.**

matrix[i][j] = 1 if there is an edge between vertex i and vertex j.
matrix[i][j] = 0 if there is no edge.



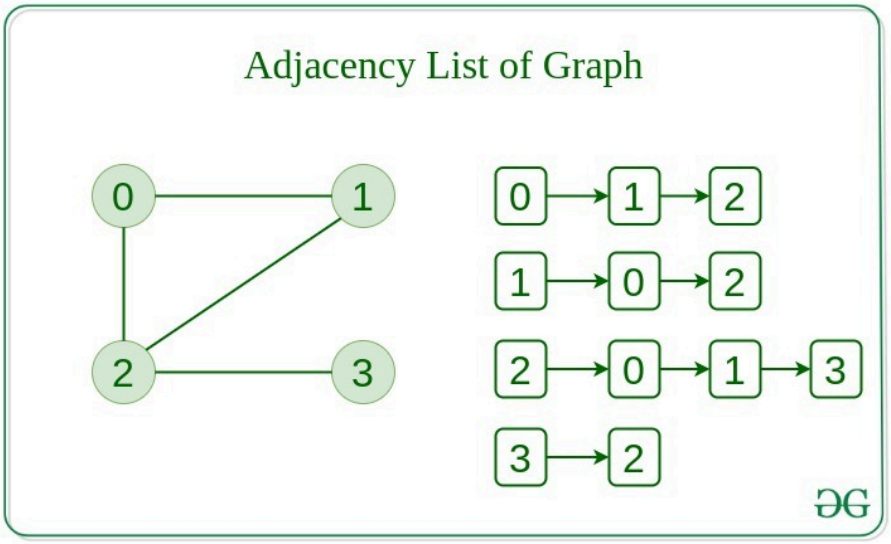

```python
class Graph:

    def __init__(self,vertex):
        self.mat=[[0]*vertex for _ in range(vertex)]
        self.size=vertex


    def add_edge(self,src,dest):
        if 0<= src < self.size and 0<= dest < self.size:

            self.mat[src][dest]=1
            self.mat[dest][src]=1

        else:
            print("Invalid Edge")


    def print(self):
        for row in self.mat:
            print(' '.join(map(str,row)))

G=Graph(5)
G.add_edge(0,1)
G.add_edge(0,2)
G.add_edge(1,3)
G.add_edge(2,4)
G.add_edge(3,4)
G.add_edge(2,3)

G.print()
```

Output

```
kali% /usr/bin/python /home/faris/grph.py
0 1 1 0 0
1 0 0 1 0
1 0 0 1 1
0 1 1 0 1
0 0 1 1 0
```

Adjacency List Representation of Graph:

This graph is represented as a **collection of array lists**. There is **an array of pointer which points to the edges connected to that vertex.**



Indegree

The indegree of a vertex (node) is the number of edges coming into that vertex.

Indegree(v)=Number of incoming edges to v

Outdegree

The outdegree of a vertex is the number of edges going out of that vertex.

Outdegree(v)=Number of outgoing edges from v

```python
class Graph:

    def __init__(self):
        self.adjList={}


    def add_vertex(self,vertex):
        if vertex not in self.adjList:
            self.adjList[vertex]=[]


    def addEdge(self,src,dest):
        self.add_vertex(src)
        self.add_vertex(dest)


        self.adjList[src].append(dest)
        self.adjList[dest].append(src)

    def printGraph(self):
        for vertex in self.adjList:
            print(vertex," -> " , self.adjList[vertex] ,end="\n")



G= Graph()
G.addEdge(1,2)
G.addEdge(2,3)
G.addEdge(1,4)
G.addEdge(4,3)
G.addEdge(2,4)
G.addEdge(4,5)
G.addEdge(3,5)

G.printGraph()
```

```
kali% /usr/bin/python /
1  ->  [2, 4]
2  ->  [1, 3, 4]
3  ->  [2, 4, 5]
4  ->  [1, 3, 2, 5]
5  ->  [4, 3]
```

Find Indegree and Outdegree

```python
def find_degrees(adj_list):
    n = len(adj_list)

    # Initialize in-degree list
    in_degree = [0] * n

    # Out-degree is simply the length of adjacency list
    out_degree = [0] * n

    # Calculate degrees
    for i in range(n):
        out_degree[i] = len(adj_list[i])  # Out-degree

        for vertex in adj_list[i]:       # In-degree
            in_degree[vertex] += 1

    # Print results
    for i in range(n):
        print(f"Vertex {i} -> In-degree: {in_degree[i]}, Out-degree: {out_degree[i]}")


# Main function
def main():
    adjacency = [
        [3, 4],       # 0 → 3, 4
        [3, 2],       # 1 → 3, 2
        [],           # 2 → none
        [2, 4],       # 3 → 2, 4
        [2, 3],       # 4 → 2, 3
        [1, 4, 6],    # 5 → 1, 4, 6
        [5]           # 6 → 5
    ]

    find_degrees(adjacency)


if __name__ == "__main__":
    main()
```

```
kali% /usr/bin/python /home/faris/grph.py
Vertex 0 -> In-degree: 0, Out-degree: 2
Vertex 1 -> In-degree: 1, Out-degree: 2
Vertex 2 -> In-degree: 3, Out-degree: 0
Vertex 3 -> In-degree: 3, Out-degree: 2
Vertex 4 -> In-degree: 3, Out-degree: 2
Vertex 5 -> In-degree: 1, Out-degree: 3
Vertex 6 -> In-degree: 1, Out-degree: 1
kali%
```

Graph Traversal DFS

DFS from a Given Source in Directed Graph

Depth-First Search (DFS) from a given source is a method of exploring a directed graph by starting at a **specific vertex** and **traversing each node as far as we can go down in the path.** If we reach a **vertex that has no unvisited neighbors,** we **backtrack to the previous vertex to explore any other paths that haven't been visited yet.** This approach is particularly useful for tasks such as **finding paths, checking connectivity, and exploring all reachable nodes from a starting point.**

How It Works:

- Keep a boolean visited array to keep track of which vertices have already been visited. This will help in not entering in infinite loops when there are cycles in the graph.
- Use Recursion to visited all unvisited neighbours of source node:
    - First marks the current vertex as visited and processes the current vertex (for example, by printing its value).
    - Then, recursively visits each unvisited neighbor of the current vertex.
    - If a vertex has no unvisited neighbors, backtracks to the previous vertex to explore other unvisited paths.

Code Style	Graph Type
mat[src][dest] = 1 only	Directed
mat[src][dest] = 1 and mat[dest][src] = 1	Undirected

Example For Undirected Graph


```python
class Graph:

    def __init__(self, vertex):
        self.mat = [[0] * vertex for _ in range(vertex)]
        self.size = vertex

    def add_edge(self, src, dest):
        if 0 <= src < self.size and 0 <= dest < self.size:
            self.mat[src][dest] = 1
            self.mat[dest][src] = 1   # Undirected graph
        else:
            print("Invalid Edge")

    def printGraph(self):
        for row in self.mat:
            print(' '.join(map(str, row)))
```

```python
    def dfs(self, src):
        visited = [False] * self.size
        stack = [src]

        while stack:
            v = stack.pop()

            if not visited[v]:
                print(v, end=" -> ")
                visited[v] = True

                # Push neighbors
                for i in range(self.size - 1, -1, -1):  # reverse order (optional)
                    if self.mat[v][i] == 1 and not visited[i]:
                        stack.append(i)
```

```python
# Create Graph
G = Graph(6)
G.add_edge(0,1)
G.add_edge(0,2)
G.add_edge(2,3)
G.add_edge(2,4)
G.add_edge(3,5)
G.add_edge(4,5)

print("DFS Traversal:")
G.dfs(0)
```

Output=>DFS Traversal:

0 -> 1 -> 2 -> 3 -> 5 -> 4 ->

DFS of a Disconnected Graph

In a disconnected graph, some vertices may not be reachable from a single source. To ensure all vertices are visited in DFS traversal, we iterate through each vertex, and if a vertex is unvisited, we perform a DFS starting from that vertex being the source. This way, DFS explores every connected component of the graph.

A disconnected graph is a graph in which not all vertices are reachable from a single starting vertex.

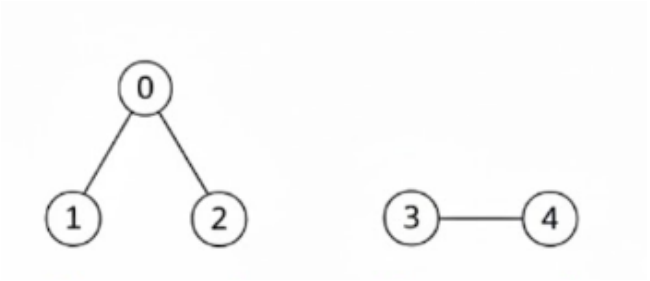
It contains multiple connected components.

Example:

Component 1: 0 → 1 → 2
Component 2: 3 → 4
Component 3: 5 → 6 **So this graph has 3 disconnected components.**

Algorithm for DFS of a Disconnected Graph

1. Create a visited[] array and mark all vertices as False.
2. Loop through all vertices from 0 to V-1.
3. If a vertex is not visited:
    - Call DFS from that vertex.
4. Repeat until all vertices are visited.



Component 1 Component 2 Component 3

```python
class Graph:

    def __init__(self, vertex):
        self.mat = [[0] * vertex for _ in range(vertex)]
        self.size = vertex

    def add_edge(self, src, dest):
        if 0 <= src < self.size and 0 <= dest < self.size:
            self.mat[src][dest] = 1
            self.mat[dest][src] = 1   # Undirected graph
        else:
            print("Invalid Edge")

    def printGraph(self):
        print("Adjacency Matrix:")
        for row in self.mat:
            print(' '.join(map(str, row)))
```

```python
    # DFS for disconnected graph
    def dfs_all(self):
        visited = [False] * self.size

        for start in range(self.size):
            if not visited[start]:
                print("\nStart from:", start)
                stack = [start]

                while stack:
                    v = stack.pop()

                    if not visited[v]:
                        print(v, end=" -> ")
                        visited[v] = True

                        # Reverse order to maintain natural DFS order
                        for i in range(self.size - 1, -1, -1):
                            if self.mat[v][i] == 1 and not visited[i]:
                                stack.append(i)
```

```python
# Create Disconnected Graph
G = Graph(7)

# Component 1
G.add_edge(0, 1)
G.add_edge(0, 2)

# Component 2
G.add_edge(3, 4)

# Component 3
G.add_edge(5,6)

G.printGraph()

print("\nDFS Traversal of Disconnected Graph:")
G.dfs_all()
print()
```

Output:

```
kali% /usr/bin/python /home/faris/grph.py
Adjacency Matrix:
0 1 1 0 0 0 0
1 0 0 0 0 0 0
1 0 0 0 0 0 0
0 0 0 0 1 0 0
0 0 0 1 0 0 0
0 0 0 0 0 0 1
0 0 0 0 0 1 0

DFS Traversal of Disconnected Graph:

Start from: 0
0 -> 1 -> 2 ->
Start from: 3
3 -> 4 ->
Start from: 5
5 -> 6 ->
kali%
```

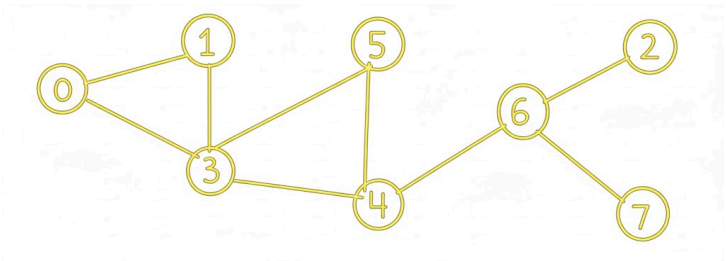
Graph Traversal BFS

Breadth First Search (BFS) is a graph traversal algorithm that starts from a source node and explores the graph level by level. First, it visits all nodes directly adjacent to the source. Then, it moves on to visit the adjacent nodes of those nodes, and this process continues until all reachable nodes are visited.

- **BFS is different from DFS in a way that closest vertices are visited before others. We mainly traverse vertices level by level.**
- Popular graph algorithms like **Dijkstra's shortest path, Kahn's Algorithm, and Prim's algorithm are based on BFS.**
- BFS itself can be used to detect cycle in a directed and undirected graph, find shortest path in an unweighted graph and many more problems.

BFS from a Given Source in an Undirected Graph:

The algorithm starts from a given source vertex and explores all vertices reachable from that source, visiting nodes in increasing order of their distance from the source, level by level using a queue. Since graphs may contain cycles, a vertex could be visited multiple times. To prevent revisiting a vertex, a visited array is used.

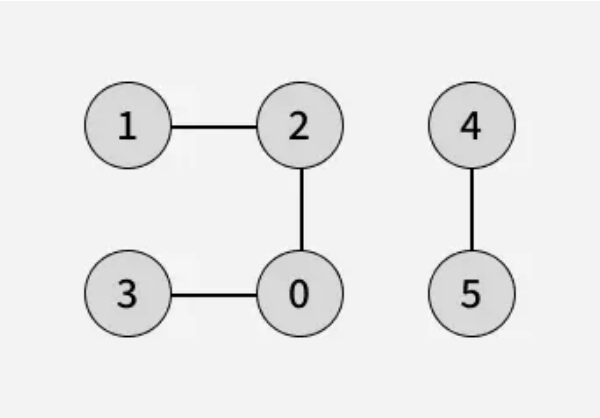


```python
from collections import deque


class Graph:

    def __init__(self,vertex):
        self.mat=[[0] * vertex for _ in range(vertex)]
        self.size=vertex


    def add_edge(self,src,dest):
        if 0 <= src < self.size and 0 <= dest < self.size:
            self.mat[src][dest]=1
            self.mat[dest][src]=1
        else:
            print("Invalid Edge")


    def bfs(self,src):
        visited=[False] * self.size
        queue=deque([src])
        visited[src]=True

        while queue:
            v= queue.popleft()
            print(v,end=' ')

            for i in range(self.size):
                if self.mat[v][i]==1 and visited[i] == False:
                    visited[i]=True
                    queue.append(i)

        print()



G=Graph(8)
G.add_edge(0,1)
G.add_edge(0,3)
G.add_edge(1,3)
G.add_edge(3,5)
G.add_edge(3,4)
G.add_edge(4,5)
G.add_edge(4,6)
G.add_edge(6,2)
G.add_edge(6,7)
G.bfs(0)
```

Output: 0 1 3 4 5 6 2 7

BFS of a Disconnected Undirected Graph:

In a disconnected graph, some vertices may not be reachable from a single source. To ensure all vertices are visited in BFS traversal, we iterate through each vertex, and if any vertex is unvisited, we perform a BFS starting from that vertex being the source. This way, BFS explores every connected component of the graph



From the image, the graph has 6 vertices (0–5) and the following edges:

- 1 — 2
- 2 — 0
- 0 — 3
- 4 — 5

So there are two disconnected components:

- Component 1: 1–2–0–3
- Component 2: 4–5

Note

bfs() traverses a single connected component starting from a source node, while bfs_all() ensures traversal of all components in a disconnected graph by repeatedly calling BFS on unvisited nodes.

```python
from collections import deque

class Graph:

    def __init__(self, vertex):
        self.mat = [[0] * vertex for _ in range(vertex)]
        self.size = vertex

    def add_edge(self, src, dest):
        if 0 <= src < self.size and 0 <= dest < self.size:
            self.mat[src][dest] = 1
            self.mat[dest][src] = 1
        else:
            print("Invalid Edge")


    # BFS for entire graph (handles disconnected graph)
    def bfs_all(self):
        visited = [False] * self.size

        for start in range(self.size):
            if not visited[start]:
                queue = deque([start])
                visited[start] = True

                while queue:
                    v = queue.popleft()
                    print(v, end=' ')

                    for i in range(self.size):
                        if self.mat[v][i] == 1 and not visited[i]:
                            visited[i] = True
                            queue.append(i)
        print()


# Graph with 6 vertices (0–5)
G = Graph(6)

G.add_edge(1, 2)
G.add_edge(2, 0)
G.add_edge(0, 3)
G.add_edge(4, 5)

print("BFS of Disconnected Graph:")
G.bfs_all()
```

Output:0 2 1 3 4 5

Time Complexity: O(V + E), The for loop ensures BFS starts from every unvisited vertex to cover all components, but the visited array ensures each vertex and edge is processed only once, keeping the total time complexity to be linear.

Auxiliary Space: O(V), using a queue to keep track of the vertices that need to be visited.

Difference between Tree and Graph:

Tree is a restricted type of Graph Data Structure, just with some more rules. **Every tree will always be a graph but not all graphs will be trees**. Linked List, Trees, and Heaps all are special cases of graphs.

Feature	Graph	Tree
Definition	A collection of nodes (vertices) and edges, where edges connect nodes.	A hierarchical data structure consisting of nodes connected by edges with a single root node.
Structure	Can have cycles (loops) and disconnected components	No cycles; connected structure with exactly one path between any two nodes.
Root Node	No root node; nodes may have multiple parents or no parents at all.	Has a designated root node that has no parent.
Node Relationship	Relationships between nodes are arbitrary.	Parent-child relationship; each node (except the root) has exactly one parent.
Edges	Each node can have any number of edges.	If there is n nodes then there would be n-1 number of edges
Traversal Complexity	Traversal can be complex due to cycles and disconnected components.	Traversal is straightforward and can be done in linear time.
Application	Used in various scenarios like social networks, maps, network optimization, etc.	Commonly used for hierarchical data representation like file systems, organization charts, HTML DOM, XML documents, etc.
Examples	Social networks, road networks, computer networks.	File systems, family trees, HTML DOM structure.

Real-Life Applications of Graph Data Structure:

Graph Data Structure has numerous real-life applications across various fields. Some of them are listed below:

- **Social Networks:** Represent users and their connections; used to find mutual friends, suggest new connections, and detect communities.
- **Computer Networks:** Model routers and data links; used for efficient routing, fault detection, and network optimization.
- **Transportation Networks:** Represent cities and routes; used to find shortest or fastest paths and plan optimal travel routes.
- **Neural Networks:** Represent neurons and synapses; used to simulate learning, brain behavior, and data processing.
- **Compilers:** Represent data dependencies and control flows; used for optimization, register allocation, and code analysis.
- **Robot Path Planning:** Represent states and transitions; used to compute the safest or shortest route for autonomous movement.
- **Project Dependencies:** Represent tasks and dependencies; used in topological sorting to determine the correct execution order.
- **Network Optimization:** Represent network nodes and links; used to minimize cost, reduce latency, and improve efficiency.

Advantages of Graph Data Structure:

- **Graphs are flexible:** Unlike arrays, linked lists, or trees, graphs have no restrictions and can represent any type of relationship.
- **Model real-world problems:** Useful for pathfinding, data clustering, network analysis, and machine learning.
- **Represent items and relationships:** Any set of items and their connections can be modeled as a graph.
- **Simplifies complex data:** Graphs make complex relationships easy to visualize and understand.